

Deep learning theory: Approximation

Ganghua Wang

1 A general picture of learning theory

We use the notation from the survey paper Gühring et al. (2020).

Assume that $\mathcal{X}$ is an *input space* and $\mathcal{Y}$ a *target space*, $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \rightarrow [0, \infty]$ is a *loss function* and $\mathbb{P}_{(\mathcal{X}, \mathcal{Y})}$ a (usually unknown) *probability distribution* on some σ -algebra of $\mathcal{X} \times \mathcal{Y}$. We then aim at finding a minimizer of the risk functional³

$$\mathcal{R} : \mathcal{Y}^{\mathcal{X}} \rightarrow [0, \infty], f \mapsto \int_{\mathcal{X} \times \mathcal{Y}} \mathcal{L}(f(x), y) d\mathbb{P}_{(\mathcal{X}, \mathcal{Y})}(x, y),$$

induced by $\mathcal{L}$ and $\mathbb{P}_{(\mathcal{X}, \mathcal{Y})}$ (where $\mathcal{Y}^{\mathcal{X}}$ denotes the set of all functions from $\mathcal{X}$ to $\mathcal{Y}$). That means we are looking for a function $\hat{f}$ with

$$\hat{f} = \operatorname{argmin} \{ \mathcal{R}(f) : f \in \mathcal{Y}^{\mathcal{X}} \}.$$

In practice, to make the optimization feasible

1. We need to restrict to a smaller function space $\mathcal{H}$

$$\hat{f}_{\mathcal{H}} = \operatorname{argmin} \{ \mathcal{R}(f) : f \in \mathcal{H} \}.$$

2. We use the empirical risk from finite sample since the underlying distribution $\mathbb{P}_{\mathcal{X}, \mathcal{Y}}$ is unknown

$$\hat{f}_{\mathcal{H}, \mathcal{S}} = \operatorname{argmin} \{ \mathcal{R}_{\mathcal{S}}(f) : f \in \mathcal{H} \}$$

3. We computationally find a solution $\hat{f}_{\mathcal{H}, \mathcal{S}}^* \in \mathcal{H}$ as the approximation of the global min.

The total error can be decomposed as follows.

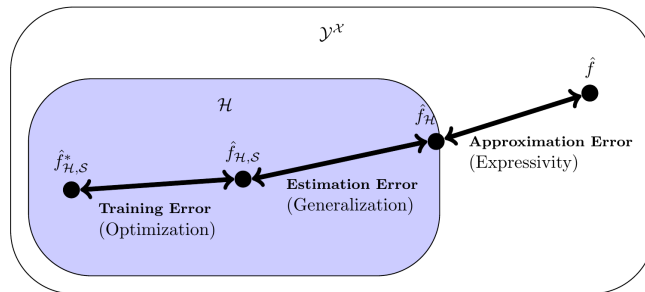


Figure 1: Decomposition of the overall error into training error, estimation error and approximation error

The approximation theory of neural networks are best understood among those three errors.

2 Neural networks

Definition 1.1. Let $d, s, L \in \mathbb{N}$. A neural network Φ with input dimension d , output dimension s and L layers is a sequence of matrix-vector tuples

$$\Phi = ((\mathbf{W}^{[1]}, \mathbf{b}^{[1]}), (\mathbf{W}^{[2]}, \mathbf{b}^{[2]}), \dots, (\mathbf{W}^{[L]}, \mathbf{b}^{[L]})),$$

where $n_0 = d$, $n_L = s$ and $n_1, \dots, n_{L-1} \in \mathbb{N}$, and where each $\mathbf{W}^{[\ell]}$ is an $n_\ell \times n_{\ell-1}$ matrix, and $\mathbf{b}^{[\ell]} \in \mathbb{R}^{n_\ell}$. If Φ is a neural network as above, $K \subset \mathbb{R}^d$, and if $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ is arbitrary, then we define the associated realization of Φ with activation function σ over K (in short, the σ -realization of Φ over K) as the map $R_\sigma(\Phi): K \rightarrow \mathbb{R}^s$ such that

$$R_\sigma(\Phi)(x) = \mathbf{x}^{[L]},$$

where $\mathbf{x}^{[L]}$ results from the following scheme:

$$\begin{aligned} \mathbf{x}^{[0]} &:= x, \\ \mathbf{x}^{[\ell]} &:= \sigma(\mathbf{W}^{[\ell]} \mathbf{x}^{[\ell-1]} + \mathbf{b}^{[\ell]}), \quad \text{for } \ell = 1, \dots, L-1, \\ \mathbf{x}^{[L]} &:= \mathbf{W}^{[L]} \mathbf{x}^{[L-1]} + \mathbf{b}^{[L]}, \end{aligned}$$

and where σ acts componentwise, that is, $\sigma(v) = (\sigma(v_1), \dots, \sigma(v_m))$ for every $v = (v_1, \dots, v_m) \in \mathbb{R}^m$.

We call $N(\Phi) := d + \sum_{j=1}^L n_j$ the number of neurons of the neural network Φ and $L = L(\Phi)$ the number of layers. For $\ell \leq L$ we call $M_\ell(\Phi) := \|\mathbf{W}^{[\ell]}\|_0 + \|\mathbf{b}^{[\ell]}\|_0$ the number of weights in the ℓ -th layer and we define $M(\Phi) := \sum_{\ell=1}^L M_\ell(\Phi)$, which we call the number of weights of Φ . Finally, we denote by $\max\{N_1, \dots, N_{L-1}\}$ the width of Φ .

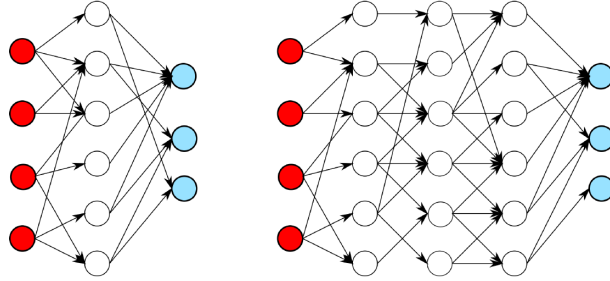


Figure 2: Visualization of a (left) shallow and a (right) deep (feedforward) neural network

3 Shallow networks

3.1 UAT

The history dates back to the Kolmogorov (Kolmogorov, 1957), and UAT has various versions given different function classes and activation functions.

Theorem 1 (Universal approximation theorem, informal). *For any compact set $\mathcal{K} \in \mathcal{X}$, family of two-layer neural networks is dense in $\mathcal{C}(\mathcal{K})$.*

The proofs are mostly based on constructions. Note that this theorem only states the capacity of NN, but we don't know the rate of approximation. In other words, how large do we need to have an ϵ -close approximation for any function f ?

3.2 Complexity bounds

It turns out the complexity will depend on the smoothness of f . Assume that the closeness is evaluated by ℓ_p norm and f belongs to the Sobolev space with degree n and unit Sobolev norm, then

Theorem 2 (Lower bound, informal). *For any $\epsilon > 0$ and f , if there exist a width N NN, then $N \gtrsim \epsilon^{-(d-1)/n}$.*

Theorem 3 (Upper bound, informal). *For any $\epsilon > 0$ and f , there exist a width N NN such that $N \lesssim \epsilon^{-d/n}$.*

Note that some proofs are actually inaccurate and doubtful. Need a more careful inspection of conditions and assumptions if you want to use any of them, though they are generally believed to be true.

4 Deep networks

Theorem 4.1 ([126] [36]). *For every $f \in F_{n,d}^p$ there exists a neural network $\Phi_{f,\epsilon}$ whose realization is capable of approximating f with error ϵ in $W^{r,p}$ -norm ($0 \leq r \leq 1$), with*

$$(i) \ M(\Phi_{f,\epsilon}) \lesssim \epsilon^{-d/(n-r)} \cdot \log_2^2(\epsilon^{-n/(n-r)}), \text{ and}$$

$$(ii) \ L(\Phi_{f,\epsilon}) \lesssim \log_2(\epsilon^{-n/(n-r)}).$$

Take home message: the capacity of deep networks are the same as shallow networks. They both suffer from the curse of dimensionality. However, under some assumptions, the deeper networks are preferred and the complexity can be greatly reduced.

Usually, we need the function has some structure. Examples are

1. Hierarchical function.

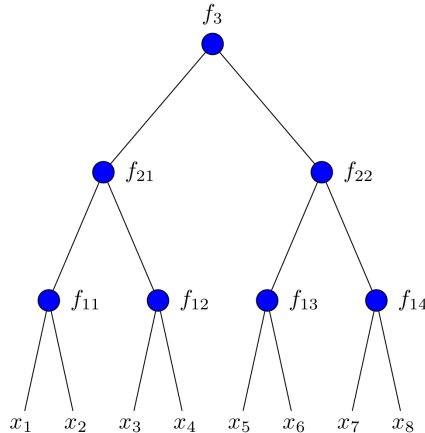


Figure 3: A visualization of the hierarchically structured function.

2. Function f has a low-dimension embedding.



Figure 4: Visualization of the Swiss Roll Data Manifold .

3. It is the solution of some PDEs. A deep enough NN can be regarded as the discrete time/space approximation of certain PDEs.

References

Ingo Gühring, Mones Raslan, and Gitta Kutyniok. Expressivity of deep neural networks. *arXiv preprint arXiv:2007.04759*, 2020.

Andrei Nikolaevich Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. In *Doklady Akademii Nauk*, volume 114, pages 953–956. Russian Academy of Sciences, 1957.